SWC3

Software-Entwicklung mit C und C++

Erik Pitzer

WS 2026/27

FH OÖ - Campus Hagenberg - Software Engineering



7. C Anhang

Anhang A – C Header & Hilfsfunktionen (i)

- Hilfsfunktionen der Folien wurden in `swc3.h` implementiert und in allen Beispielen (wo nicht anders angegeben) inkludiert

```c
#include <stdio.h>   // printf
#include <limits.h>  // integer value limits
#include <float.h>   // float value limits
#include <stdint.h>  // fixed-size integers
#include <stddef.h>
#include <stdarg.h>  // variadic functions
#include <ctype.h>   // character types
#include <complex.h> // complex numbers
#include <stdlib.h>  // malloc, free, ...
#include <time.h>
#include <tgmath.h> // type-generic math
```

Anhang A – C Header & Hilfsfunktionen (ii)

```c
// adapted from https://stackoverflow.com/a/17290414
#define TYPENAME(x) _Generic((x),                                                 \
         bool: "bool",                   unsigned char: "unsigned char",          \
         char: "char",                     signed char: "signed char",            \
    short int: "short int",         unsigned short int: "unsigned short int",     \
          int: "int",                     unsigned int: "unsigned int",           \
     long int: "long int",           unsigned long int: "unsigned long int",      \
long long int: "long long int", unsigned long long int: "unsigned long long int", \
        float: "float",                         double: "double",                 \
  long double: "long double",                   char *: "pointer to char",        \
       void *: "pointer to void",                int *: "pointer to int",         \
   _Decimal32: "_Decimal32",            float _Complex: "float _Complex",         \
   _Decimal64: "_Decimal64",           double _Complex: "double _Complex",        \
  _Decimal128: "_Decimal128",     long double _Complex: "long double _Complex",   \
 int(*)(void): "function",                                                        \
 int(*)(const char *restrict, ...): "library function",                           \
      default: "other")
```

Anhang A – C Header & Hilfsfunktionen (iii)

```c
// NOTE: formatting of _Complex is a hack specific to GCC
#define PRINTF_FORMAT_CHAR(X) _Generic((X),                                 \
          bool: "%d",                    unsigned char: "%hhu",     \
          char: "'%c'",                    signed char: "%hhd",     \
     short int: "%hd",             unsigned short int: "%hu",      \
           int: "%d",                    unsigned int: "%u",       \
      long int: "%ld",              unsigned long int: "%lu",      \
 long long int: "%lld",        unsigned long long int: "%llu",     \
         float: "%g",                           double: "%g",      \
   long double: "%Lg",                          char *: "\"%s\"",  \
        void *: "0x%x",                          int *: "0x%x",    \
    _Decimal32: "%Hf",                 float _Complex: "%g%+gi",   \
    _Decimal64: "%Df",                double _Complex: "%g%+gi",   \
   _Decimal128: "%DDf",           long double _Complex: "%Lg%+Lgi", \
      int(*)(): "%p",                                               \
 int(*)(const char *restrict, ...): "%p",                           \
       default: "0x%x")
```

```c
#define SHOW_VALUE(X) printf(PRINTF_FORMAT_CHAR(X), (X))
#define SHOW(X)         \
  printf("%s = ", #X); SHOW_VALUE(X); printf("\n")
```

Anhang A – C Header & Hilfsfunktionen (iv)

```c
#define SHOW2(X, Y)                       \
  printf("%s = ", #X); SHOW_VALUE(X);     \
  printf(";  %s = ", #Y);  SHOW_VALUE(Y); \
  printf("\n")
```

```c
#define SHOW_XY(P)      \
  printf("%s = (", #P); \
  SHOW_VALUE((P).x);    \
  printf(", ");         \
  SHOW_VALUE((P).y);    \
  printf(")\n")
```

```c
#define SHOW_TYPE(X)              \
  printf("%s = ", #X);            \
  SHOW_VALUE(X);                  \
  printf(" (%s)\n", TYPENAME(X))
```

```c
#define SHOW_COMPLEX(X) printf("%s = %g+%gi\n", #X, creal(X), cimag(X))
```

Anhang A – C Header & Hilfsfunktionen (v)

```c
#define SHOW_ARRAY_N(A, N)                          \
  printf("%s = {", #A);                             \
  for (size_t i = 0; i<(N); i++) {                  \
    if (i > 0) printf(", ");                        \
    printf(PRINTF_FORMAT_CHAR((A)[i]), ((A)[i])); \
  }                                                 \
  printf("}\n")
```

```c
#define SHOW_POINTS_N(A, N)        \
  printf("%s = {", #A);            \
  for (size_t i = 0; i<(N); i++) { \
    if (i > 0) printf(", ");       \
    printf("(");                   \
    SHOW_VALUE((A)[i].x);          \
    printf(",");                   \
    SHOW_VALUE((A)[i].y);          \
    printf(")");                   \
  }                                \
  printf("}\n")
```

```c
#define SHOW_DBL(X) printf("    double: %.25f\n", (X))
#define SHOW_D32(X) printf("_Decimal32: %.25Hf\n", (X))
```

Anhang A – C Header & Hilfsfunktionen (vi)

```c
#define SHOW_NUMBER(N) printf("%s = {.f=% 4.2f, .i=0b%032b}\n", #N, (N).f, (N).i);

#define SHOW_FLOAT(F) printf("%s = {.value=% 4.2f .sign=%c .exp=0x%02x .mant=0x%06x}\n", \
                             #F, (F).value, (F).sign == 0 ? '+' : '-', (F).exp, (F).mant);
```

```c
#include <locale.h>

#define SHOW_RANGE(name, min, max, format)                      \
  printf("=%s= \t %2zu \t %'25" #format " \t %'25" #format "\n", \
         #name, sizeof(name)*8, min, max)

#define SHOW_RANGE_S(name, abbr, format) SHOW_RANGE(name, abbr ## _MIN, abbr ## _MAX, format)
#define SHOW_RANGE_U(name, abbr, format) SHOW_RANGE(name,            0, abbr ## _MAX, format)
```

```c
#define SHOW_HEX_CHARS(C)                                       \
  for (size_t i = 0; i<sizeof((C)); ++i) {                      \
    if (i > 0 && i % 4 == 0) printf("\n");                      \
    uint8_t c = (C)[i];                                         \
    printf("%2zu: 0x%02x", i, c);                               \
    if (isprint(c)) printf(" '%c'", c); else printf("  ? ");   \
    printf(" | ");                                              \
  }
```

Anhang B – C Compiler-Parameter

- C Beispiele in den Folien wurden (falls nicht anders angegeben) mit folgendem Kommando übersetzt

```
gcc -std=c23 -pedantic \
 -Wall -Wextra -Wconversion -Wshadow \
 -fsanitize=address,leak,undefined -ftrivial-auto-var-init=pattern  \
 -lasan -lubsan -lm -ldfp \
 -o example.exe example.c
```

- Zusätzlich wurde unsichtbar bei jedem Beispiel der Header `swc.h` inkludiert

```
#include "swc.h"
```